



(Semana 13): Proyecto Integrador U4 - Avance 13/16: Uso de bases de datos relacionales: configuración, modelos y consultas básicas

Apertura: lunes, 7 de septiembre de 2026, 07:00

Cierre: domingo, 13 de septiembre de 2026, 23:59



Tarea Semana 13

Proyecto Integrador U4 - Avance 13/16: Uso de bases de datos relacionales: configuración, modelos y consultas básicas

Continuidad del Proyecto Integrador

El estudiante deberá continuar trabajando sobre el mismo **Proyecto Integrador** desarrollado durante las semanas anteriores. En la **Semana 12** se implementó persistencia local utilizando **SQLite**, integrando los formularios Flask-WTF con operaciones de almacenamiento y consulta.

Durante la **Semana 13**, la aplicación deberá evolucionar hacia el uso de una **base de datos relacional MySQL o PostgreSQL**. Las funcionalidades previamente desarrolladas para **agregar, listar y modificar registros** deberán adaptarse para trabajar directamente con la nueva base de datos. Además, se deberá incorporar o comprobar la funcionalidad para **eliminar registros**, completando el flujo básico de gestión de datos.



Contexto y objetivo de la tarea

Esta tarea corresponde a la **Semana 13** de la asignatura **Desarrollo de Aplicaciones Web** y forma parte de la **Unidad 4: Gestión de Datos y Desarrollo de la Aplicación Web**. El objetivo es que el estudiante configure la conexión entre Flask y una base de datos relacional, modele las principales entidades del sistema mediante tablas, establezca relaciones utilizando claves primarias y foráneas e integre las funcionalidades existentes de la aplicación con operaciones SQL. Como mínimo, uno de los módulos del Proyecto Integrador deberá permitir **consultar, agregar, modificar y eliminar registros** utilizando **MySQL o PostgreSQL** como fuente real de datos.

Actividad a desarrollar

El estudiante deberá configurar una base de datos relacional utilizando **MySQL o PostgreSQL** y conectarla con el Proyecto Integrador desarrollado en Flask. Deberá reutilizar los formularios, rutas y plantillas construidos en las semanas anteriores y adaptarlos para que las operaciones de gestión de datos se ejecuten directamente sobre la nueva base de datos. Como mínimo, un módulo del sistema deberá permitir **listar, agregar, modificar y eliminar registros**. Para un sistema de ferretería, se recomienda implementar completamente estas operaciones sobre el módulo de **Productos** y mantener preparados los módulos de Clientes, Proveedores y Facturación.

Motor de base de datos

El estudiante podrá utilizar uno de los siguientes motores:

Opción A - MySQL

Instalar el conector:

```
pip install mysql-connector-python
```

Opción B - PostgreSQL

Instalar el conector:

```
pip install psycopg2-binary
```

Para mantener uniformidad con los ejemplos trabajados en clase se recomienda utilizar **MySQL**. No obstante, el estudiante podrá utilizar PostgreSQL si mantiene correctamente la estructura y funcionalidades solicitadas.



Estructura esperada del proyecto

```
Proyecto XXXXXXXX/
|
|— app.py
|— requirements.txt
|
|— conexion/
|   |— __init__.py
|   |— conexion.py
|
|— sql/
|   |— esquema.sql
|
|— forms/
|   |— __init__.py
|   |— producto_form.py
|   |— cliente_form.py
|   |— proveedor_form.py
|   |— facturacion_form.py
|
|— templates/
|   |— base.html
|   |— index.html
|
|   |— productos.html
|   |— formulario_producto.html
|
|   |— clientes.html
|   |— formulario_cliente.html
|
|   |— proveedores.html
|   |— formulario_proveedor.html
|
|   |— facturacion.html
|   |— formulario_facturacion.html
|
|   |— components/
|       |— navbar.html
|       |— footer.html
|
|— static/
|   |— css/
|       |— style.css
|   |— js/
|       |— script.js
```

```
└─ img/  
   └─ imágenes del proyecto
```

La conexión deberá centralizarse dentro de **conexion/conexion.py**. El archivo **sql/esquema.sql** deberá permitir crear nuevamente la estructura de la base de datos.

Modelo relacional mínimo

El estudiante deberá identificar las principales entidades del Proyecto Integrador y representarlas mediante tablas relacionadas. En un proyecto de ferretería, por ejemplo:

Productos

id_producto (PK), nombre, precio, stock, id_proveedor (FK)

Proveedores

id_proveedor (PK), nombre, teléfono, correo

Clientes

id_cliente (PK), nombre, cédula, teléfono, correo

Facturas

id_factura (PK), id_cliente (FK), fecha, total

Las entidades y campos deberán adaptarse a la temática de cada Proyecto Integrador.

Flujo de gestión de datos que deberá comprobar

LISTAR

SELECT

AGREGAR

INSERT

MODIFICAR

UPDATE

ELIMINAR

DELETE

Las cuatro operaciones deberán probarse utilizando directamente **MySQL o PostgreSQL**. No deberán simularse mediante listas o diccionarios de Python.

☰ Instrucciones para desarrollar la tarea

- Continúe trabajando sobre el mismo Proyecto Integrador desarrollado hasta la **Semana 12**.
- No elimine los formularios, validaciones, rutas, plantillas, componentes ni estilos desarrollados anteriormente.
- Instale y configure **MySQL o PostgreSQL**.
- Cree una base de datos correspondiente a la temática de su proyecto.
- Configure la conexión desde Flask mediante el archivo **conexion/conexion.py**.
- Compruebe que Flask pueda establecer correctamente la conexión con la base de datos.
- Actualice **requirements.txt** después de instalar el conector correspondiente.
- Defina las tablas principales del Proyecto Integrador.
- Cada tabla deberá disponer de una **PRIMARY KEY**.
- Establezca al menos una relación utilizando **FOREIGN KEY**.
- Guarde las instrucciones **CREATE TABLE** dentro de **sql/esquema.sql**.
- Seleccione como mínimo un módulo para implementar completamente las operaciones de consulta, registro, modificación y eliminación.
- En el proyecto de ferretería se recomienda utilizar el módulo **Productos**.
- Modifique la ruta de listado para obtener los registros directamente desde MySQL o PostgreSQL mediante una consulta **SELECT**.
- Utilice **fetchall()** para recuperar varios registros.
- Muestre los registros recuperados mediante Jinja2 dentro de una tabla HTML.
- La funcionalidad **Agregar** deberá utilizar el formulario Flask-WTF desarrollado previamente.
- Mantenga **form.validate_on_submit()** antes de insertar información.
- Realice el registro mediante una consulta **INSERT INTO**.
- Ejecute **commit()** después de registrar información.

- Pruebe que el nuevo registro aparezca inmediatamente en el listado.
- La funcionalidad **Modificar** deberá recuperar primero el registro seleccionado utilizando su identificador.
- Cargue los datos actuales dentro del formulario de edición.
- Permita modificar los datos utilizando el mismo formulario o un formulario específico para edición.
- Guarde los cambios utilizando una consulta **UPDATE ... SET ... WHERE**
- Utilice siempre la cláusula **WHERE** para modificar únicamente el registro seleccionado.
- Ejecute **commit()** después de actualizar.
- Pruebe que la información modificada aparezca correctamente en el listado.
- Incorpore o adapte la funcionalidad **Eliminar**.
- Elimine únicamente el registro seleccionado utilizando una consulta **DELETE FROM ... WHERE**
- Utilice siempre **WHERE** en la operación DELETE para evitar eliminar todos los registros de la tabla.
- Ejecute **commit()** después de eliminar información.
- Se recomienda incorporar una confirmación visual antes de eliminar definitivamente un registro.
- Pruebe que el registro eliminado desaparezca del listado y también de la base de datos.
- Utilice consultas SQL parametrizadas en **INSERT, UPDATE, SELECT y DELETE**.
- No concatene directamente en una consulta SQL los valores recibidos desde los formularios.
- Cierre correctamente los cursores y conexiones después de realizar las operaciones.
- Implemente al menos una consulta utilizando **WHERE**.
- Implemente al menos una consulta relacionada entre dos tablas, utilizando una clave foránea.
- Puede utilizar **JOIN** para mostrar información relacionada.
- Mantenga el uso de **base.html**, herencia Jinja2 y componentes reutilizables.
- Mantenga los recursos CSS, JavaScript e imágenes dentro de **static**.
- Mantenga Bootstrap para la presentación de formularios, tablas, botones y mensajes.
- Ejecute localmente la aplicación Flask.

- Compruebe que las operaciones **Agregar, Modificar y Eliminar** funcionen realmente sobre MySQL o PostgreSQL.
- Reinicie Flask y compruebe que los cambios realizados permanezcan almacenados.
- Revise directamente la información desde MySQL Workbench, pgAdmin u otra herramienta del motor seleccionado.
- Suba todos los cambios realizados al repositorio de **GitHub**.
- Mantenga actualizada la publicación visual mediante **GitHub Pages**.

Pruebas obligatorias del avance

Antes de realizar la entrega, el estudiante deberá comprobar obligatoriamente el siguiente flujo:

1. Inicie la aplicación Flask y acceda al módulo seleccionado.
2. Compruebe que el listado se obtenga desde **MySQL o PostgreSQL**.
3. **Agregue un nuevo registro** mediante el formulario.
4. Compruebe que el registro fue almacenado realmente en la base de datos.
5. **Modifique el registro** previamente creado.
6. Compruebe directamente en la base de datos que los valores fueron actualizados.
7. **Elimine un registro**.
8. Compruebe que el registro dejó de existir tanto en la interfaz web como en la base de datos.
9. Cierre y ejecute nuevamente Flask.
10. Compruebe que todos los cambios realizados se mantienen correctamente.

Operaciones SQL mínimas a evidenciar

SELECT

Mostrar los registros almacenados en la base de datos.

INSERT

Agregar un nuevo registro desde un formulario validado.

UPDATE

Modificar un registro existente utilizando su identificador.

DELETE

Eliminar únicamente el registro seleccionado.

Elementos mínimos que debe evidenciar el proyecto**Base de datos relacional**

MySQL o PostgreSQL configurado y funcionando.

Conexión Flask

La aplicación deberá conectarse correctamente a la base de datos.

Modelo relacional

Tablas con claves primarias y al menos una clave foránea.

Listar

Visualización de registros mediante SELECT y Jinja2.

Agregar

Registro mediante formulario Flask-WTF e INSERT.

Modificar

Edición de un registro mediante UPDATE.

Eliminar

Eliminación mediante DELETE utilizando el identificador.

Persistencia

Los cambios deberán permanecer después de reiniciar Flask.

Parámetros de entrega

Formato**GitHub y GitHub Pages****Evidencia****Flask + MySQL/PostgreSQL****Fecha de entrega****Según cronograma de Moodle****Producto esperado**

El estudiante deberá entregar el **enlace del repositorio de GitHub** y el **enlace de GitHub Pages** del Proyecto Integrador actualizado. El repositorio deberá evidenciar la configuración y conexión de una base de datos **MySQL o PostgreSQL**, el modelo relacional del sistema y las funcionalidades para **listar, agregar, modificar y eliminar registros**. Como mínimo, un módulo deberá ejecutar correctamente las operaciones **SELECT, INSERT, UPDATE y DELETE** sobre la base de datos relacional. Las funcionalidades deberán reutilizar los formularios Flask-WTF, validaciones, rutas, plantillas Jinja2 y componentes desarrollados en las semanas anteriores.

Importante sobre GitHub Pages

GitHub Pages únicamente permitirá visualizar la parte frontend estática del Proyecto Integrador. La aplicación Flask y las operaciones **SELECT, INSERT, UPDATE y DELETE** con MySQL o PostgreSQL deberán comprobarse localmente. GitHub deberá contener todo el código backend, los formularios, la configuración de conexión y el archivo **sql/esquema.sql**.

Recomendaciones

- Continúe utilizando el mismo Proyecto Integrador.
- No elimine las funcionalidades desarrolladas anteriormente.
- Centralice la conexión con la base de datos.
- No suba contraseñas reales de MySQL o PostgreSQL al repositorio público.
- Utilice consultas parametrizadas.
- Mantenga las validaciones Flask-WTF.
- Utilice siempre **WHERE** en UPDATE y DELETE.
- Utilice **commit()** después de INSERT, UPDATE y DELETE.
- Compruebe directamente en la base de datos cada operación realizada.
- Mantenga actualizado **requirements.txt**.

- Mantenga disponible **sql/esquema.sql**.

Antes de entregar el avance deberá ejecutar obligatoriamente una prueba completa: **agregar un registro, modificarlo y posteriormente eliminarlo**, comprobando cada cambio tanto desde la aplicación Flask como directamente desde MySQL o PostgreSQL.

Proyecto Integrador U4 - Avance 13/16

Esta actividad corresponde al **Avance 13/16 del Proyecto Integrador**. El trabajo desarrollado deberá integrarse con los avances anteriores, especialmente con los formularios, validaciones y persistencia desarrollados durante las Semanas 11 y 12. **Este avance no se calificará de manera independiente**. La calificación se realizará al finalizar el **Proyecto Integrador**, considerando la integración progresiva del frontend, Flask, Jinja2, Flask-WTF, validaciones y bases de datos relacionales, así como el correcto funcionamiento de las operaciones de consulta, registro, modificación y eliminación de datos.

✦ ✦ Características de IA ▾

Estado de la entrega

Número del intento	Este es el intento 1.
Estado de la entrega	Esta tarea no requiere que usted envíe nada de forma online
Estado de la calificación	Sin calificar
Tiempo restante	8 horas 34 minutos restante
Última modificación	-
Comentarios de la entrega	> Comentarios (0)

Actividad previa

(Semana 13) Actividad de Rebote: Uso de bases de datos relacionales: configuración, modelos y consultas básicas.

Ir a...

Siguiente actividad

Evaluación: Semana 13: Uso de bases de datos relacionales: configuración, modelos y consultas básicas.

Contáctanos



Síguenos



Usted se ha identificado como JOHAO ALEXI CAICEDO BAUTISTA (Cerrar sesión)

Resumen de retención de datos

Descargar la app para dispositivos móviles

Descargar la app para dispositivos móviles

Este tema fue desarrollado por

conecti.me